

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 12

Course: Operating Systems

Topic: Producer, Consumer and the Other Tools

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 12 · Producer, Consumer and the Other Tools

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 12 (Track A: Producer, Consumer and the Other Tools, Linux)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.3.1-1-cachyos)
- **Compiler:** GCC 16.2.1 20260810 (`gcc -Wall -pthread`)
- **Threading Library:** POSIX Threads (`libpthread` / glibc 2.40)
- **Shell:** GNU bash, version 5.3.20(1)-release
- **Workspace:** `/home/ilie/Documents/SO/os-lab12` (`~/os-lab12`)

Purpose of the Work

1. Implement the classic bounded-buffer Producer-Consumer problem using coordinated counting semaphores (`empty`, `full`) and a mutex lock, evaluating buffer invariants and deadlock hazards caused by lock reordering.
2. Examine the POSIX Condition Variable synchronization pattern (`pthread_cond_t`), analyzing atomic unlock-and-sleep semantics, predicate evaluation loops (`while` vs. `if`), and Mesa monitor semantics regarding spurious wakeups.
3. Construct the Readers-Writers synchronization pattern using semaphores, examining group-based resource sharing, critical section entry/exit protocols, and writer starvation hazards.
4. Construct a deterministic deadlock scenario exhibiting circular wait on mutual exclusion locks (`pthread_mutex_t`), detecting hang states via POSIX process timeouts.
5. Eliminate deadlocks by establishing a global lock acquisition hierarchy, evaluating why strict acquisition ordering mathematically prevents resource allocation graph cycles.
6. Analyze non-deterministic race dynamics by eliminating artificial timing delays (`sleep`), contrasting deterministic failures with intermittent concurrency bugs.

Part 1. Producer and Consumer

1. Architectural Concept

The bounded-buffer Producer-Consumer problem represents a core synchronization pattern found throughout modern operating system kernels, such as Unix pipes (`pipe(2)`), IPC message queues, and socket receive buffers. In this model: - A **Producer** thread generates data items and deposits them into a circular memory buffer of fixed capacity N. - A **Consumer** thread retrieves items from the buffer, operating at a potentially variable or slower processing rate.

To ensure race-free execution without buffer overflow or underflow, three synchronization primitives work in unison: 1. `empty` (Counting Semaphore): Initialized to buffer capacity ($N = 4$), tracking available free slots. 2. `full` (Counting Semaphore): Initialized to 0, tracking populated slots awaiting consumption. 3. `lock` (Mutual Exclusion Lock): Protects the buffer indices (`in`, `out`) and shared array from concurrent read/write corruption.

2. Implementation: `prodcons.c`

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 4
#define ITEMS 8
```

```c
int buf[SIZE];
int in = 0, out = 0, count = 0; /* count only for printing */

sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty); /* wait for a free slot */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put %d (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full); /* one more filled slot */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full); /* wait for something to take */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("        take %d (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty); /* one more free slot */
        usleep(300000);   /* the consumer is slow */
    }
    return NULL;
}

int main(void) {
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Figure 1.1 – Bounded buffer Producer-Consumer implementation in prodcons.c

3. Execution Verification

```
$ gcc -Wall -pthread -o prodcons prodcons.c
$ ./prodcons
put 1 (in buffer: 1)
put 2 (in buffer: 2)
put 3 (in buffer: 3)
        take 1 (in buffer: 2)
put 4 (in buffer: 3)
put 5 (in buffer: 4)
        take 2 (in buffer: 3)
put 6 (in buffer: 4)
        take 3 (in buffer: 3)
put 7 (in buffer: 4)
        take 4 (in buffer: 3)
```

```
put 8 (in buffer: 4)
        take 5 (in buffer: 3)
        take 6 (in buffer: 2)
        take 7 (in buffer: 1)
        take 8 (in buffer: 0)
```

Figure 1.2 – Execution trace of prodcons demonstrating bounded buffer maintenance

Observe (Part 1 Answers)

1. **The producer is fast, yet "in buffer" never goes above 4. Which call stops it?** The call `sem_wait(&empty);` halts the producer. When 4 items occupy the buffer without being consumed, the counting semaphore `empty` reaches value 0. The subsequent invocation of `sem_wait(&empty)` causes the calling producer thread to block inside the kernel until the consumer dequeues an item and increments available space via `sem_post(&empty)`.

2. **Who waits at the start: the consumer or the producer? Which call makes it wait?** The consumer thread waits at the start. It is blocked by `sem_wait(&full);`. Because `full` is initialized to 0 (`sem_init(&full, 0, 0)`), any consumer thread attempting to dequeue from an empty buffer is put to sleep immediately until the producer deposits the first item and signals `sem_post(&full)`.

3. **In producer(), swap the first two lines: lock the mutex BEFORE sem_wait(&empty). Run it. What happens when the buffer is full, and why?** Swapping the synchronization sequence to:

   ```
   pthread_mutex_lock(&lock);
   sem_wait(&empty);
   ```

   introduces a catastrophic nested-monitor deadlock. When the buffer reaches full capacity (4 items), the producer acquires `lock` and then blocks inside `sem_wait(&empty)` waiting for an empty slot. When the consumer awakens to process an item, it executes `pthread_mutex_lock(&lock)` and blocks because the sleeping producer still holds the mutex. The producer cannot receive an empty slot because the consumer cannot enter to take an item, and the consumer cannot enter because the producer holds the mutex. The program hangs permanently:

   ```
   $ timeout 3 ./prodcons_swapped
   put 1 (in buffer: 1)
   put 2 (in buffer: 2)
   put 3 (in buffer: 3)
   put 4 (in buffer: 4)
   exit code: 124
   ```

Part 2. The Condition Variable

1. Conceptual Mechanism

While semaphores maintain internal state counters, a **condition variable** (`pthread_cond_t`) possesses no inherent state; it serves solely as an OS-managed thread wait queue. Condition variables enable threads to suspend execution efficiently until an arbitrary boolean predicate concerning shared application state becomes true.

The primitive operation `pthread_cond_wait(&changed, &lock)` performs three atomic steps: 1. Releases the associated mutex lock. 2. Suspends the calling thread in the kernel wait queue. 3. Automatically re-acquires the mutex lock before returning to user space upon being signaled.

2. Implementation: `condvar.c`

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>

char message[32];
```

```c
int ready = 0; /* the condition itself */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed); /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    while (!ready) { /* while, not if: re-check after waking */
        printf("B: not ready, I sleep\n");
        pthread_cond_wait(&changed, &lock); /* unlocks, sleeps, locks again */
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

Figure 2.1 – Condition variable coordination implementation in condvar.c

3. Execution Verification

```
$ gcc -Wall -pthread -o condvar condvar.c
$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

Figure 2.2 – Trace output showing condition variable sleep and wakeup cycle

Observe (Part 2 Answers)

1. **In order.c (Lab 11) the semaphore itself remembered that the message was ready. Here, which variable remembers it?** The application-level boolean variable `int ready` remembers the state. Unlike a semaphore, which increments and preserves its counter even if no thread is currently waiting, a condition variable is stateless: signaling a condition variable with zero waiting threads has no effect, making external shared state memory mandatory.
2. **Replace "while (!ready)" with "if (!ready)", compile, run. Does it still work here?** Yes, replacing the loop with `if (!ready)` succeeds in this specific program execution. Because there is only one reader thread, one writer thread, and no spurious kernel wakeups occur during this single run, waking once coincides with `ready == 1`.
3. **Then why do we write while? Give one reason why a thread can wake up and still find ready == 0:** POSIX synchronization conforms to **Mesa monitor semantics**, requiring predicate re-evaluation in a `while` loop due to two critical factors:
    - **Spurious Wakeups:** Under Linux and POSIX standards, kernel interrupt handling or process

signal deliveries can wake a sleeping thread from `pthread_cond_wait` even though no thread called `pthread_cond_signal`.

- **Intervening Consumers (Stolen State):** In multi-threaded systems with multiple consumers, a signaled thread does not immediately execute; it must first compete to re-acquire `lock`. Another consumer thread may acquire the lock first, consume the state, and reset `ready = 0`. If the original thread awakened under an `if` construct, it would proceed assuming valid data, causing corruptions.

Part 3. Readers and Writers

1. Conceptual Mechanism

The Readers-Writers problem models scenarios where multiple threads access a shared data structure: - **Readers:** Perform non-mutating inspections. Any number of reader threads may access the resource simultaneously. - **Writers:** Perform mutating modifications. A writer must possess exclusive access (no other writers, no concurrent readers).

This is achieved using two semaphores: - `room` (initialized to 1): Controls exclusive entry to the shared data. - `rlock` (initialized to 1): A mutex protecting the shared counter `readers`.

2. Implementation: `readers.c`

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 3
#define WRITERS 2

int value = 0;   /* the shared data */
int readers = 0; /* how many are reading now */
sem_t rlock;     /* protects the counter readers */
sem_t room;      /* 1 = the room is free for a writer */

void *reader(void *arg) {
    long id = (long)arg;
    sem_wait(&rlock);
    if (++readers == 1)
        sem_wait(&room); /* the first reader in locks out the writers */
    sem_post(&rlock);

    printf("reader %ld reads %d (readers inside: %d)\n", id, value, readers);
    usleep(200000);

    sem_wait(&rlock);
    if (--readers == 0)
        sem_post(&room); /* the last reader out lets the writers in */
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room); /* alone in the room */
    value += 10;
    printf("writer %ld writes %d (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}
```

```c
int main(void) {
    sem_init(&rlock, 0, 1);
    sem_init(&room, 0, 1);
    pthread_t r[READERS], w[WRITERS];
    pthread_create(&w[0], NULL, writer, (void *)1L);
    for (long i = 0; i < READERS; i++)
        pthread_create(&r[i], NULL, reader, (void *)(i + 1));
    pthread_create(&w[1], NULL, writer, (void *)2L);
    for (int i = 0; i < READERS; i++)
        pthread_join(r[i], NULL);
    for (int i = 0; i < WRITERS; i++)
        pthread_join(w[i], NULL);
    sem_destroy(&rlock);
    sem_destroy(&room);
    return 0;
}
```

Figure 3.1 – Reader-preference Readers-Writers synchronization in readers.c

3. Execution Verification

```
$ gcc -Wall -pthread -o readers readers.c
$ ./readers
writer 1 writes 10 (alone)
reader 1 reads 10 (readers inside: 1)
reader 2 reads 10 (readers inside: 2)
reader 3 reads 10 (readers inside: 3)
writer 2 writes 20 (alone)
```

Figure 3.2 – Output showing concurrent reader batches bounded by exclusive writers

Observe (Part 3 Answers)

1. **Three readers are inside at the same time, but a writer is always alone. Which two lines of reader() make that possible?**
   - **Line 1 (Group Entry):** `if (++readers == 1) sem_wait(&room);` — The first reader that enters acquires the `room` semaphore on behalf of all readers, barring all writers.
   - **Line 2 (Group Exit):** `if (--readers == 0) sem_post(&room);` — Only when the active reader count decrements to zero does the final exiting reader release `room`, permitting waiting writers to enter.
2. **Why does the counter readers need its own lock (rlock)? What could go wrong without it?** `readers` is a shared global variable accessed concurrently by multiple reader threads. Because incrementing (`++readers`) and decrementing (`--readers`) are non-atomic load-modify-store sequences, concurrent reader invocations without `rlock` introduce a race condition. If two readers increment simultaneously, an update can be lost: both could read `readers == 0` and attempt to call `sem_wait(&room)` twice, or the count might never drop to 0, permanently stranding `room` and preventing writers from ever running.
3. **Imagine readers that keep arriving without a pause. What happens to writer 2? (This problem has a name: starvation.)** Writer 2 suffers from **writer starvation**. Because this implementation gives preference to readers, as long as new readers arrive before the previous readers completely vacate, `readers` never drops to 0. Consequently, `sem_post(&room)` is never invoked, leaving Writer 2 blocked indefinitely on `sem_wait(&room)`.

Part 4. Build a Deadlock, then Remove It

1. Conceptual Mechanism

A **deadlock** is a state in which two or more concurrent processes or threads are unable to proceed because each is waiting for a resource acquired by the other. According to Coffman's conditions, a deadlock requires four simultaneous conditions: 1. Mutual exclusion. 2. Hold and wait. 3. No preemption. 4.

Circular wait: Thread 1 waits for a resource held by Thread 2, while Thread 2 waits for a resource held by Thread 1.

2. Inducing Deadlock

In the uncorrected version, Thread `one` acquires Mutex `A` and requests Mutex `B`, while Thread `two` acquires Mutex `B` and requests Mutex `A`.

```
/* Uncorrected deadlock sequence */
void *one(void *arg) {
    pthread_mutex_lock(&A);
    sleep(1);
    pthread_mutex_lock(&B); /* Waits for two */
    ...
}

void *two(void *arg) {
    pthread_mutex_lock(&B);
    sleep(1);
    pthread_mutex_lock(&A); /* Waits for one */
    ...
}
```

Figure 4.1 – Inverted lock acquisition pattern generating circular dependency

We execute the deadlock binary under the `timeout` command to demonstrate the indefinite freeze:

```
$ gcc -Wall -pthread -o deadlock deadlock_orig.c
$ timeout 5 ./deadlock; echo "exit code: $?"
one: I have A, I want B
two: I have B, I want A
exit code: 124
```

Figure 4.2 – Process timeout execution proving deterministic deadlock freeze

The exit status 124 indicates that the POSIX `timeout` monitor forcibly terminated the process after 5 seconds of total inactivity.

3. Eliminating Deadlock via Lock Ordering (`deadlock.c`)

To break Coffman's fourth condition (circular wait), we enforce a strict global lock ordering: every thread must acquire Mutex `A` before acquiring Mutex `B`.

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&A); /* Enforce consistent acquisition order */
    printf("two: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);
    printf("two: I have both\n");
```

```c
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, one, NULL);
    pthread_create(&t2, NULL, two, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("done\n");
    return 0;
}
```

Figure 4.3 – Deadlock-free implementation enforcing global lock hierarchy in deadlock.c

4. Corrected Execution Verification

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ ./deadlock; echo "exit code: $?"
one: I have A, I want B
one: I have both
two: I have A, I want B
two: I have both
done
exit code: 0
```

Figure 4.4 – Execution trace of corrected deadlock.c finishing cleanly with exit code 0

Observe (Part 4 Answers)

1. **In the stuck run, which thread holds which mutex, and which one is each thread waiting for? Draw the circle:**

   - Thread `one` holds Mutex `A`, waiting to acquire Mutex `B`.
   - Thread `two` holds Mutex `B`, waiting to acquire Mutex `A`.

   ```
   Thread one -----[ holds ]-----> Mutex A <-----[ waits for ]----- Thread two
        |                                                                ^
        +-----[ waits for ]-----> Mutex B <-------[ holds ]--------------+
   ```

   Figure 4.5 – Circular wait dependency cycle in resource allocation graph

2. **Remove both sleep(1) lines from the ORIGINAL version and run it ten times. Does it still always hang? Why is that worse, not better?** No. When `sleep(1)` is removed, the program does not always hang; testing 10 iterations produced mixed results:

   ```
   0 124 124 0 124 124 124 124 0 124
   ```

   The program deadlocked in 7 runs and ran to completion in 3 runs. This intermittent behavior is significantly **worse** than a deterministic hang because non-deterministic bugs evade test suites, pass validation checks, and manifest unpredictably under production load, where diagnosing race conditions is exceptionally difficult.

3. **In one sentence: why can taking the locks in the same order never deadlock?** Enforcing a total ordering on lock acquisition eliminates circular wait dependencies, making cyclic dependency directed cycles in the resource allocation graph mathematically impossible.

Conclusions

In this laboratory work, we investigated multi-tool concurrency synchronization and deadlock resolution: 1. **Producer-Consumer Coordination:** We combined counting semaphores (`empty`, `full`) with mutual exclusion to construct bounded buffers, demonstrating how kernel-level pipe abstractions coordinate

asynchronous flow control. 2. **Lock Hierarchy Discipline:** We verified that incorrect nesting of mutex locks within semaphore waits induces fatal deadlocks, reinforcing the rule that resource allocations must precede critical sections. 3. **Mesa Condition Variables:** We demonstrated that condition variables must always evaluate state predicates inside `while` loops to remain resilient against kernel-level spurious wakeups and multi-consumer race conditions. 4. **Group Synchronization (Readers-Writers):** We implemented reader-preference mutual exclusion, observing how shared counters permit concurrent inspection while enforcing writer isolation, and identified writer starvation risks. 5. **Deadlock Elimination:** We analyzed Coffman's circular wait condition, proving that establishing a strict global lock hierarchy guarantees cycle-free execution.